

Building AI Agents with LangGraph: An Intermediate Developer's Handbook

Welcome to the **LangGraph** developer's handbook. This comprehensive guide is organized into ten modules, each focused on a core aspect of building AI agents using **LangGraph**. The content is tailored for developers with Python experience and familiarity with LLM frameworks like LangChain, aiming to deepen your skills in constructing robust, stateful AI agent systems. Throughout, we'll use a practical, developer-friendly tone and provide clear explanations, code examples, visuals, best practices, and exercises to reinforce learning. Let's get started!

1. Foundations of Agentic Graphs with LangGraph

Objectives: In this module, you will learn what LangGraph is and why it was created. We'll cover the limitations of basic loop-based agents, and how LangGraph introduces **agentic graphs** – cyclic workflows that empower more complex, flexible agent behavior ¹ ². You'll become familiar with LangGraph's key primitives (nodes, edges, and state) and understand how they form a **stateful graph** that underpins an agent's reasoning process ³ ⁴.

LangChain's Agent Executor vs. LangGraph

Before LangGraph, LangChain's standard *agent executor* loop handled decisions and tool use in a fixed pattern. This design was powerful but rigid – every agent followed the same step-by-step loop with limited flexibility ¹. For simple Q&A or single-turn tasks, that might suffice, but complex, **multi-step workflows** demanded more adaptability. LangGraph was introduced to address these limitations ⁵.

LangGraph is an advanced library in the LangChain ecosystem that enables *cyclic* decision processes ⁶. Unlike LangChain's linear directed acyclic graph (DAG) chains, LangGraph workflows can include **cycles** – meaning an agent can loop through reasoning steps multiple times, revisit tools, and make dynamic decisions as conditions change ⁷ ⁸. This cyclic capability is essential for **agentic behavior**, where an AI agent may need to reflect, backtrack, or iterate on a plan ⁸.

Key idea: LangGraph treats an agent's decision-making flow as a *graph* instead of a fixed loop. Nodes in the graph represent operations (LLM calls, tool calls, etc.), and edges represent possible transitions or control flow. Crucially, cycles in the graph allow an agent to re-enter a prior step based on new information or updated state ⁷ ⁹. This unlocks more complex behaviors like retrying after an error, refining a query, or multi-turn planning that adapts over time.

Core Concepts: Nodes, Edges, and State

At the heart of LangGraph are three core primitives:

- **Nodes:** Each node is a unit of work – for example, calling an LLM with a prompt, performing a calculation, or invoking a tool. A node can wrap any callable (Python function or LangChain *Runnable*) that takes some input state and returns some output ¹⁰. You might have a node for

the AI's thought process, another node for executing tool calls, etc. In essence, nodes are the graph's **vertices**, encapsulating logic or actions.

- **Edges:** Edges define **control flow** between nodes, i.e. which node leads to which next node ¹¹. Edges can be static (always go from Node A to Node B) or conditional. With **conditional edges**, you can embed branching logic: the agent's output at one node can determine whether it goes to a tool node, loops back, or ends the process ¹² ¹³. Edges thus allow both linear sequences and dynamic decision points in the graph.
- **State:** The state is a shared context or memory that flows through the graph. In LangGraph, state is typically represented as a Python dictionary or a TypedDict that carries information like conversation history, intermediate results, or flags between nodes ¹⁴. Every node can read from and update this **persistent state**, enabling later steps to use data from earlier steps ¹⁴. This design means the agent can "remember" what happened previously in the cycle – for example, which tools have been used or what the last user message was. The state evolves as the agent proceeds, making the graph **stateful**.

How it works: When an agent runs, it starts with an **initial state** (e.g. the user's query). The state is passed into the starting node of the graph (often an LLM node). That node produces an output and updates the state (for instance, adding the LLM's response into a message list). Then, an edge directs the flow to the next node based on that output. This continues: each node consults and updates the state, edges route to the appropriate next step, and so on ¹⁵ ¹⁶. Eventually, a termination condition is met (an edge leads to an **END** node or no further edge is taken), and the agent stops with a final state/result.

Why Graphs Enable Agentic Behavior

Using graphs (with cycles) instead of a simple loop brings several benefits:

- **Dynamic looping:** The agent isn't forced into a fixed number of tool uses or steps. It can loop *as needed*, stopping when a goal is satisfied or continuing if there's more to do. For example, an agent can keep using a search tool until it finds a sufficient answer, rather than a fixed one-and-done approach ⁵ ⁸.
- **Conditional branching:** Different situations can trigger different paths. If an LLM decides no tool is needed, the graph can go straight to answering; if tools are needed, the graph can branch to a tool node ¹² ¹³. This makes the agent's workflow **context-dependent and adaptable**, rather than always following the same sequence.
- **Persistent context:** Because state is carried through, the agent has **memory of prior turns** or steps within the same session. This is crucial for multi-turn conversations or multi-step problem solving, where each step builds on the last. Traditional stateless function calls would lose that context.
- **Transparency and debugging:** Representing the workflow as a graph gives developers a clearer mental model of the agent's logic. You can visualize the nodes and edges, which helps in understanding and debugging how the agent reaches decisions. (We'll see in Module 7 how LangGraph lets you inspect and even visualize these graphs for development insight.)

Example scenario: Imagine a customer service agent using LangGraph. A user asks, “What are the advantages of solar panels?” The agent starts at a node that decides how to answer. It might realize it needs more info (like the user’s location or electricity bill) and loop into a *question-asking* node to gather that info from the user ¹⁵. The state is updated with the user’s answers. Then an edge condition might decide to invoke a *calculation tool* node (for energy cost savings) if relevant ⁹. The tool’s result updates the state, and the agent loops back to consider if more steps are needed or if it can now provide a final answer ¹⁶. This cycle can repeat – ask user, use tool, analyze – until the agent has enough info to conclude. Finally, an edge leads to an end state where the agent produces the answer. This *feedback loop* of decision, action, and state update is exactly what LangGraph is designed for ⁹.

LangGraph Architecture at a Glance

LangGraph is inspired by prior distributed and graph computing frameworks like Google’s Pregel and Apache Beam ⁸. If you’re familiar with **NetworkX** (a Python graph library), LangGraph’s interface will feel similar in how you define nodes/edges. Under the hood, LangGraph provides the runtime to execute this graph: iterating over nodes, managing the state, and handling loops until completion.

Some key features and components include:

- **StateGraph class:** a builder for defining your workflow. You add nodes and edges to a `StateGraph` and then compile it into an executable agent.
- **Prebuilt Executors:** LangGraph includes ready-made templates for common agent patterns (like a ReAct agent loop, which we’ll explore later). These let you get started quickly by instantiating a typical graph (LLM + Tool loop) with one call ¹⁷ ¹⁸.
- **Chat Agent State:** LangGraph provides a `MessagesState` (a list of messages) to conveniently represent conversation state for chat-based agents ¹⁹. This integrates with LangChain’s message objects, making it easy to maintain dialogue history.

By the end of this module, you should grasp why LangGraph’s graph-based approach is powerful for building *production-ready agents*. In the next modules, we’ll dive deeper into implementing these concepts: how to model control flow explicitly, integrate tools, maintain memory, and more.

Suggested Exercise: Sketch a simple agent workflow as a graph. Identify at least 3 nodes (e.g., an LLM node to decide action, a tool node, and an output node) and draw arrows for edges (including any conditional branch). Explain in a few sentences how the state changes as your agent moves through the graph. This exercise will help solidify your understanding of agentic graphs.

2. Modeling State & Control Flow

Objectives: This module delves into how to explicitly model **state** and **control flow** in LangGraph. You will learn how to define a custom `State` schema, use the `StateGraph` API to add nodes and edges, and implement conditional branching logic. We’ll walk through a code example constructing a basic agent workflow step by step, highlighting how the agent’s state is passed and transformed. By the end, you should be comfortable building your own graph (with loops and branches) to control an agent’s reasoning process.

Defining the Agent’s State Schema

A well-defined state is fundamental to your agent’s operation. In LangGraph, you can define a state as a Python dict or, for clarity, a `TypedDict` (or Pydantic model) listing the fields your agent will track. For instance, if you are building a math problem-solving agent, your state might include

`{"question": ..., "working": ..., "answer": ...}` to hold the problem, the step-by-step working, and the final answer.

For **chat agents**, LangGraph often uses `MessagesState`, which is essentially `{"messages": [...list of Message objects...]}` capturing the conversation history ¹⁹. This is convenient because LangChain message objects (`HumanMessage`, `AIMessage`, `ToolMessage`, etc.) can be stored here, and many LangGraph tools/LLM integrations expect the state in this format.

Example: Let's say we want to model an agent that either answers directly or uses a tool. We'll keep a state with a list of messages. We can use the provided `MessagesState` from `langgraph.graph`:

```
from langgraph.graph import MessagesState
state = MessagesState() # initially empty
```

This `MessagesState` acts like a dict with one key `"messages"` that holds a list of tuples or message objects.

For a more complex agent, you might extend the state. For example, if an agent needs to keep track of a user profile or some session info, you could define:

```
from typing_extensions import TypedDict

class MyAgentState(TypedDict):
    messages: list # conversation messages
    user_profile: dict # some info about the user
    last_tool_used: str # name of the last tool invoked (if any)
```

When building the `StateGraph`, you will specify this as the state schema: `StateGraph(MyAgentState)`.

Building a StateGraph: Adding Nodes

To create a workflow, instantiate a `StateGraph` with your state type. Then use `add_node(name, function)` to add each step of logic:

- **LLM Node:** A node that calls an LLM to decide on an action. For instance, this node might examine the current state (conversation) and either produce an answer or a tool request. In LangGraph, you can integrate LLM calls via LangChain's chat models. A simple LLM node function might take `state["messages"]` and return an updated state containing the AI's response message.
- **Tool Node:** A node to execute tool calls. LangGraph offers a `ToolNode` that, given a list of tool functions, will handle any tool invocation requests present in the state (more on tools in Module 3). The `ToolNode` itself is added as a node to the graph.
- **Control Nodes/Functions:** Sometimes you don't have a fixed next node; you want to decide at runtime. For that, LangGraph provides *conditional edges*. You write a function that inspects the state and **returns a label** indicating the next node (or a special `END` signal). One pattern,

shown below, is to have a function that checks if the last LLM message contains a tool request; if yes, route to the tool node, otherwise end the loop 20 21 .

Code Walkthrough: Building a simple ReAct-style agent graph.

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode
from typing import Literal

# Initialize the graph builder with a MessagesState (chat history)
workflow = StateGraph(MessagesState)

# Node 1: LLM decision node
def call_model(state: MessagesState):
    messages = state["messages"]
    response = llm_with_tools.invoke(messages) # llm_with_tools is an LLM
    set up with tool usage ability
    return {"messages": [response]} # return new message from LLM as an
    update to state

workflow.add_node("LLM", call_model)
workflow.add_edge(START, "LLM") # start at the LLM node

# Node 2: Tool execution node (prebuilt)
tools = [search_web, get_weather] # assume these are defined tool functions
tool_node = ToolNode(tools)
workflow.add_node("tools", tool_node)

# Add conditional edge from LLM -> either tools or end, based on LLM output
def call_tools(state: MessagesState) -> Literal["tools", END]:
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools" # if the LLM decided to call a tool, go to tools
    node
    return END # otherwise, no tool needed, end the agent

workflow.add_conditional_edges("LLM", call_tools)
# After using tools, always go back to LLM for the next step
workflow.add_edge("tools", "LLM")

# Compile the graph into an agent
agent = workflow.compile()
```

Let's unpack what we did:

- We created a graph with two main nodes: "LLM" and "tools". The LLM node (call_model) consults the conversation and produces a new AI message (which could be an answer or a request to use a tool). The Tool node executes actual tool functions and updates the messages (with tool results).

- We set the **start** of the graph at the LLM node `22` `23`. This means when we invoke the agent, it begins by calling `call_model` on the user's input.
- We added a **conditional edge** from `"LLM"` that uses the function `call_tools` to decide next step `12` `24`. `call_tools` checks the last message: if the LLM's response contains a `tool_calls` field (meaning the model wants to use a tool), we route to the `"tools"` node; if not, we route to `END` (which terminates the agent) `20` `21`. Under the hood, LangGraph sees the return value and directs the flow accordingly.
- We also added an edge from `"tools"` back to `"LLM"` `12` `24`. This creates a loop: after a tool is executed and the state updated with the tool's output, control goes back to the LLM node. The LLM can then incorporate the new information and decide if another tool is needed or if it can answer now.

The result is a **cyclic workflow**: LLM -> (maybe tools) -> back to LLM -> ... until the LLM produces an output that requires no tool, causing the graph to exit. This is essentially how a ReAct agent operates, but here we explicitly modeled it with a graph. We have fine-grained control: for example, we could easily insert another node in between or add another branch for handling errors (like an `"error_handler"` node if a tool fails).

Running and Testing the Workflow

To run the agent, we provide an initial state input and iterate over the outputs (especially if streaming). For instance:

```
# Example query
user_question = "Will it rain in Paris today?"

# Prepare input state
input_state = {"messages": [("user", user_question)]}

# Run the agent (streaming for demonstration)
for chunk in agent.stream(input_state, stream_mode="values"):
    final_message = chunk["messages"][-1]
    final_message.pretty_print()
```

This would cause the agent to start at the LLM node with the user question. Perhaps the LLM decides it needs to use `get_weather` tool, so it returns a tool call. The conditional edge sees a tool call, routes to the `tools` node which executes `get_weather(query="Paris")`, appends the tool result to messages, and then loops back. The LLM node runs again, now with the weather info in the state, and likely produces a final answer. The conditional edge now returns `END`, and the agent stops, yielding the answer.

You can also run in one go: `final = agent.invoke(input_state)`, and then examine `final["messages"]` for the full conversation trace (including any tool interactions). The key is that our graph orchestrated the control flow cleanly.

Design Trade-offs for Control Flow

When modeling control flow, consider:

- **Granularity of nodes:** You could make one giant node that does a lot (decide and execute tools internally) or break it into finer nodes (one node per decision or action). Finer nodes give more transparency and reusability. LangGraph encourages smaller, single-responsibility nodes (like one node per tool execution, one per decision) so that you can reuse and rearrange them easily.
- **Hardcoded vs learned decisions:** In the example, we hardcoded the branch logic (if `tool_call` exists). You could also allow the LLM to explicitly output a command that indicates next steps (LangGraph supports returning a `Command` object with a `goto` field [25](#) [26](#)). That is a more advanced pattern where the LLM's output directly encodes the next node to jump to (used in complex multi-agent setups). For simple cases, a Python conditional like above is sufficient and often clearer.
- **Error handling:** By modeling flow, you can decide what happens if something goes wrong. For example, if the tool returns an error, do you loop back and ask the LLM to rephrase? Or send an apology to the user? You could add an edge for error outputs to a special node (maybe the LLM itself can detect an error in tool result and you branch on that). Thinking through these paths makes your agent more robust.
- **END conditions:** Ensure there is some condition that eventually leads to an END, to avoid infinite loops. In our design, as soon as the LLM doesn't request a tool, we end. In other designs (like multi-turn chat), you might end when the user stops the conversation or after a certain number of turns. LangGraph doesn't force an exit; it's up to your graph logic to determine when to stop.

By explicitly controlling state and transitions, you gain *determinism* and *clarity*. You know exactly what sequences are possible, which is a big advantage in testing and maintaining AI agent systems.

Suggested Exercise: Implement a custom branching in a StateGraph. For example, add a second tool and modify the LLM's decision logic to choose between two different tool nodes. Concretely: create two tool nodes (e.g., one for web search, one for a calculator). After the LLM node, use a conditional function that routes to `"search_tools"` node if the query is a question needing internet info, or to `"calc_tools"` node if it's a math problem (you might detect this by simple keyword or format). Test your agent on two inputs ("Who is the president of France?" vs "What is 12*7?") to see it taking different branches. This will help you practice building conditional flows.

3. Tools & External Actions

Objectives: In this module, we focus on integrating **tools** into your LangGraph agents. Tools are external actions or functions the agent can use – anything from web search, database queries, calculations, to calling APIs. You'll learn how **tool calling** works with LLMs, how to define custom tools, and how LangGraph executes tool calls within an agent's graph. We'll cover both the conceptual model (LLM requesting a tool) and practical implementation (using LangChain's tool interface, `ToolNode`, and `create_react_agent`). By the end, you should be able to extend your agents with new capabilities via tools and understand the trade-offs in tool design and usage.

Understanding Tool Calling in LLM Agents

Diagram: An LLM generating a tool call. The model decides to invoke a function (`multiply`) with structured arguments instead of returning a direct answer ²⁷ ²⁸.

LLMs can output special structured requests indicating a desire to use a tool. For example, rather than answering “What is 2 multiplied by 3?” directly, the model might output a **tool invocation** like: “Call tool `multiply` with arguments {a:2, b:3}.” In LangChain/LangGraph, this comes through as an `AIMessage` that includes a `tool_calls` field describing the requested tool name and parameters ²⁹ ³⁰.

Important points about tool calls:

- The **LLM decides** when to call a tool and with what inputs, based on its prompt and training. The developer defines which tools are available and how they’re described to the model, typically via a system message or function schema.
- The model’s output is just a *request*. The model itself does **not execute** the tool (LLMs can’t directly run code). Instead, the agent’s runtime (LangGraph in this case) sees the `tool_calls` in the message and knows that it must invoke those actual functions, then pass the results back into the state for the LLM to use ³¹ ³².
- Tool calls are usually **conditional** – the model will only produce them if it feels necessary. If the user asks a straightforward question answerable from knowledge, the model might just answer. But if the user asks for current weather, the model may produce a tool call to a weather API. If no tool is needed, the model’s response is a normal content message ³¹.

In practice, with LangChain’s chat models, you “enable” tool use by providing the tool specifications to the model (often via a special OpenAI function calling or similar mechanism). When enabled, the LLM can return `AIMessage(tool_calls=[...])` objects.

LangGraph takes these in stride: as we saw in Module 2, you might have a loop where after an LLM node, you check `if last_message.tool_calls` to decide going into a tool execution node ²⁰ ²¹.

Defining Tools: Prebuilt vs Custom

LangChain provides a library of **prebuilt tools** for common actions: e.g. search engines (Bing, SerpAPI, Tavily), calculators, Python REPL, database query tools, web scraping, etc. These are readily usable – you just import them and include in your agent. You can find many in LangChain’s integrations directory ³³ ³⁴. For example, `SerpAPIWrapper` can perform Google searches, `LLMMath` can evaluate math expressions, etc.

However, often you’ll want to define **custom tools** for your specific use case. LangChain (and LangGraph by extension) makes this easy via the `@tool` decorator ³⁵ ³⁶:

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Fetch current weather for a city via WeatherAPI."""
```



```

url = f"http://api.weatherapi.com/v1/current.json?key={WEATHER_API_KEY}&q={city}"
resp = requests.get(url).json()
if "location" in resp:
    return f"{resp['location']['name']}: {resp['current']['temp_c']}°C, {resp['current']['condition']['text']}"
else:
    return "Weather data not found."

```

Here, `@tool` turns the Python function into a LangChain Tool object with an input schema and documentation automatically. The docstring is used to tell the LLM what the tool does (as part of prompt). The function's parameters define the args schema. When the LLM calls `get_weather`, the runtime will execute this function with the provided city argument.

Tool design tips:

- Keep tools **idempotent and side-effect free** if possible. They should reliably return an output for given input. If a tool might fail (network issues, etc.), be prepared to handle exceptions and perhaps return an error string the LLM can understand.
- Provide a clear docstring: The LLM uses that description to decide when to use the tool. In our example, we mention “fetch current weather for a city via WeatherAPI.” The LLM will hopefully call it for queries about weather.
- Simple outputs: It's often best if tools return text or JSON that the LLM can easily parse. In the weather example, we returned a concise string. If a tool returns complex data, consider formatting it or summarizing, unless you plan to have the LLM parse it carefully.

Once you have tools defined, you need to **bind them to an LLM** so that the model knows about them and can request them. LangChain's chat models often have a way to specify available tools (for example, OpenAI's function calling or Anthropic's observation format). LangGraph provides utility to wrap an LLM with tools. For instance, `ChatOpenAI` from `langchain_openai` can take a list of tools on initialization, or you can use `langchain_core.tools` utilities to attach them.

In our earlier code, we had `llm_with_tools = ChatOpenAI(..., tools=[...])`. This yields a model that, when invoked, may return an `AIMessage` with `tool_calls` if appropriate.

Executing Tools in LangGraph

With tools defined and an LLM prepared to use them, LangGraph offers two main patterns to execute the tools when the LLM requests them:

1. **Using `ToolNode`**: This is a prebuilt graph node in LangGraph specifically designed to execute tools. You initialize `ToolNode(tools_list)`, add it to your graph, and whenever the flow goes into this node, it will look at the last message's `tool_calls`, execute each tool call in order, and append the results as `ToolMessage` objects into the state ³⁷ ³⁸. After execution, the state now contains the outcome of the tool, and you typically route back to the LLM node to continue the reasoning with this new information.

2. Using a ToolNode ensures the LLM does **not** hallucinate the result of a tool; it actually gets the real result from the function. For example, if the LLM says “I need to use `search_web` for X”, the ToolNode will run `search_web(X)` and put whatever that returns (say some text snippet or URL) into the messages.
3. ToolNode can handle multiple tool calls if the LLM makes several in one response (some advanced agents might request a chain of tool calls at once).
4. **Using `create_react_agent`:** If you want a quick setup, LangGraph provides `create_react_agent` which internally builds a two-node graph (LLM + tools) with the logic already wired (essentially what we built manually in Module 2). For example:

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(model=llm, tools=[get_weather, search_web],
                           system_prompt=system_prompt)
```

This one-liner creates an agent that will use the given `llm` (with the system prompt to instruct tool usage) and handle the loop of tool calls automatically ³⁹ ⁴⁰. Under the hood, it's doing very similar to our custom graph: an LLM node and a Tool execution node cycling. Using `create_react_agent` is great for quick prototyping or standard agent behaviors.

Under the hood: When a tool is executed via ToolNode, how does it appear to the LLM? Typically, the result is inserted into the conversation as a special message (e.g., as if the tool “said” something). For example, after `get_weather("Paris")` returns its string, the state's messages list might get `(“tool: get_weather”, “<Paris weather data>”)`. Then the LLM sees this in the next round, allowing it to use that info in its next response. This mechanism is analogous to how LangChain's ReAct loop appends “Observation” after an “Action”. LangGraph formalizes it: tool outputs are just part of the state, no different from any other message, but marked by role or metadata as coming from a tool.

Example: Tool Integration in Action

Let's revisit a concrete example with code (simplified for clarity):

```
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent, ToolNode
from langchain_core.tools import tool

# Define a simple tool
@tool
def search_web(query: str) -> str:
    """Use Tavily to search the web for the query and return a snippet."""
    results = tavily_search.invoke(query) # assume tavily_search is set up
    snippet = results[0]["snippet"] if results else "No results."
    return snippet

# Initialize an LLM that can use tools
llm = ChatOpenAI(model="gpt-3.5-turbo", tools=[search_web])
```

```
# Quick agent using prebuilt ReAct graph
agent = create_react_agent(model=llm, tools=[search_web],
                           system_prompt="You are a research assistant. Use
search_web for any unknown facts.")

# Run the agent on a question
result = agent.invoke({"messages": [{"role": "user", "content": "Who won the
2022 World Cup?"}]}))
print(result["messages"][-1]["content"])
```

When this agent runs: - The system prompt tells the LLM it has a `search_web` tool. The user asks the question. - The LLM likely doesn't have the 2022 World Cup winner memorized (or even if it does, our instruction encourages tool use), so it outputs a tool call: e.g. `ToolCall(name="search_web", args={"query": "2022 World Cup winner"})`. - LangGraph's runtime catches that and uses `ToolNode` (inside `create_react_agent`) to execute `search_web("2022 World Cup winner")`. The snippet result (maybe "France defeated Croatia in 2018, but in 2022 Argentina won the World Cup...") is added to state as a tool response. - The LLM gets control again and now has that info in the conversation. It then answers: "Argentina won the 2022 World Cup, defeating France in the final." - The final answer is returned in `result`.

Best Practices for Tools:

- **Few, well-chosen tools:** Don't overload the agent with too many tools. Each tool increases complexity for the model's decision. Provide the tools that are truly relevant to your tasks. For example, a travel assistant agent might need a Flights API and a Weather API, but not a Wikipedia search if it's not needed.
- **Tool schemas and clarity:** Use type annotations and docstrings to clearly specify what input a tool expects and what it returns. This helps the LLM avoid mistakes in calling the tool (like passing a wrong type).
- **Testing tool behavior:** Try prompt scenarios where the tool should be used and where it shouldn't. Ensure the model actually calls the tool when expected. If not, you may need to adjust the prompt (system instructions) to encourage tool use, or possibly use few-shot examples demonstrating tool use.
- **Error handling:** If a tool fails (throws exception) or returns something the model might not understand, consider catching exceptions inside the tool and returning a friendly error message. LangGraph will not crash your agent on a Python exception in a tool – if unhandled, the exception would propagate and likely abort the agent run. So it's wise to handle within the tool and perhaps inject an error notice for the LLM (which could then decide to handle it or try a different approach).
- **Security considerations:** Be cautious with tools that perform actions (like file system writes, deletion, or sending emails). Ensure your agent has appropriate guardrails (we'll discuss in Module 6) so the LLM doesn't misuse tools. For instance, if you have a tool `delete_user_account(id)`, you might want a human approval step before the agent can actually execute it.

Tool Execution in Graph Workflows

In a LangGraph workflow, tool execution typically appears as a separate node (like our `"tools"` node). Another design is possible: you could incorporate tool logic inside an LLM node by intercepting the model's output (using a *pre* or *post* hook to catch tool requests). However, the structured approach with a ToolNode is cleaner and is the recommended pattern ³². It separates concerns: the LLM node decides *what to do*, and the Tool node actually *does it*. This aligns with the principle of **separation of reasoning and execution**.

One more powerful aspect: Tools can themselves access the agent's **state or memory** if needed. For example, a tool might want to store something for long-term memory (we'll see memory in next module). LangGraph allows tools to read/write state – e.g., a tool could update `state["user_profile"]`. Advanced usage aside, simple tools usually just get their inputs and return outputs.

Suggested Exercise: Implement a new tool and integrate it into an agent. For instance, write a `@tool` `def define_word(word: str) -> str:` that returns a dictionary definition of the word (you could call an API like DictionaryAPI or simply have a small dictionary). Create an agent that can use this tool for unknown words. Test with a user question like “What does ‘quixotic’ mean?” to see if the agent correctly calls the `define_word` tool and returns the definition. This will give you practice in extending an agent's abilities with custom tools.

4. Persistence, Checkpointing & Replay

Objectives: This module explores how LangGraph provides **persistence** for agent state, enabling long-running agents and the ability to pause/resume or “rewind” execution. We'll cover what **checkpoints** are and how they're automatically created during execution, how to configure persistent storage (in-memory or database) for these checkpoints, and how to use **replay** or “time travel” features to resume an agent from a prior state ⁴¹ ⁴². By the end, you'll know how to make your agents resilient to failures (by restarting from last checkpoint) and how to debug or iterate by replaying past runs.

Durable Execution with Checkpoints

One of LangGraph's standout benefits is *durable execution*: the ability for an agent to persist through failures or long gaps by saving its state at each step ⁴³. It achieves this through **checkpointing**. A **checkpoint** captures the entire state of the graph at a particular node (and often the identifier of that node) at runtime. In simpler terms, after each node's execution, LangGraph can save the agent's state (and context about where it is in the graph) so that if needed, we can resume from that point later ⁴⁴ ⁴⁵.

How does this help?

- If your agent is running for a long time (maybe a complex multi-step job), you don't want to lose progress if a server restarts or an error occurs. With checkpoints, you can restart the agent from the last saved state rather than from scratch ⁴³.
- You can intentionally **pause** the agent for human review (as part of HITL, see Module 6) and resume after approval – this pause is basically stopping after a checkpoint.
- For debugging, you might replay from a checkpoint to see what happens if the agent took a different turn (more on this later).

LangGraph's persistence is powered by a **Checkpoint** interface. By default, if you don't specify anything, the agent run will be ephemeral (in-memory only). But you can provide a checkpoint backend: - **In-memory checkpoint** (`InMemorySaver`): Stores checkpoints in memory (good for testing, not surviving process restarts) ⁴⁶ ⁴⁷ . - **Database-backed (Postgres, Redis, etc.)**: LangGraph offers savers like `PostgresSaver` or `RedisSaver` that store checkpoints in a database, meaning they persist across sessions ⁴⁸ ⁴⁹ . For instance, `PostgresSaver` will create tables to store states (often as JSON blobs) keyed by a `thread_id` and checkpoint sequence.

Thread IDs: Each independent conversation or agent run is identified by a `thread_id`. Think of it as a session ID. When you call `agent.invoke(input, config={"configurable": {"thread_id": "123"}})`, you're telling LangGraph "this run belongs to thread 123" ⁵⁰ . If that thread has existing history in the database, the agent will load the latest state and continue; if not, it starts fresh. By reusing thread IDs, you enable *stateful conversations* (short-term memory) as we'll see in Module 5. For now, know that `thread_id` is crucial for persistence – it scopes the checkpoints to a particular session or conversation.

Enabling persistence: To save checkpoints, you compile your graph with a `checkpoint`:

```
from langgraph.checkpoint.postgres import PostgresSaver
checkpointer = PostgresSaver.from_conn_string(DB_URI)
agent = workflow.compile(checkpointer=checkpointer)
```

Make sure to call `checkpointer.setup()` once to initialize DB tables (if needed) ⁵¹ ⁵² . Now every time the agent runs, each step's state is saved to Postgres. Similarly, for Redis you'd use `RedisSaver`.

With persistence on, if your agent crashes or times out, you could later do:

```
agent.resume(thread_id="123")
```

to continue from where it left off (the agent can pick up the last checkpoint of that thread and proceed). This is particularly useful for long jobs or running agents as background tasks.

Checkpoints and Replay (Time Travel)

Beyond fault tolerance, LangGraph exposes a powerful "time travel" API that allows you to inspect and **resume from past checkpoints** intentionally ⁴¹ ⁴² . This is useful for debugging and for implementing human corrections.

Viewing history: Using the `checkpoint`, you can retrieve a list of checkpoints for a thread. For example, `agent.get_state_history("123")` might return a list of checkpoint IDs or states, each corresponding to one step in the conversation ⁵³ ⁵⁴ . Each checkpoint includes the state (e.g., messages so far) and the node it was at.

Choosing a checkpoint: Suppose an agent made a mistake at step 5 of 10. You can identify the checkpoint right before the mistake. Perhaps by examining the saved states, or by placing an "interrupt" (breakpoint) at a certain node. Once you have the checkpoint ID, you can prepare to resume from it.

Modifying state (optional): You have the option to tweak the state before resuming ⁵⁴ ⁵⁵. For instance, if the agent misunderstood something, a human could edit the state (e.g., correct a fact in the memory or remove a wrong tool output) using `agent.update_state(thread_id, checkpoint_id, new_state)`. This creates an adjusted checkpoint.

Resuming execution: Finally, you call `agent.invoke(None, config={"configurable": {"thread_id": "123", "checkpoint_id": checkpoint_to_resume}})` ⁵⁶ ⁵⁷. Passing `None` as the input (since we're not giving a new user query, just continuing) along with the same thread and the checkpoint ID tells LangGraph to continue from that point in the graph. The agent will load that checkpoint's state and proceed forward as if it never stopped – but now it branches into a new “thread” path of execution (the history for thread 123 will now have a fork).

This mechanism allows **branching experimentation**. You can try an alternate action and compare outcomes, all preserved in the checkpoint history. LangGraph ensures that resuming from a checkpoint **creates a new fork** so you don't overwrite the original trajectory ⁴¹ ⁴². You can always go back to the original path if needed.

Example use case: Let's illustrate with a joke-writing agent (like in LangGraph's time travel tutorial). The agent has two steps: generate a joke topic, then write a joke. Say it generated a topic “chickens crossing the road” and wrote a joke. If you want to see a different joke, you could resume from after the topic generation checkpoint but provide a modified topic in state to force a different joke ⁵⁸ ⁵⁶. Or even let it regenerate the topic by resuming from the very start but keep a memory of an attempted topic to avoid repeating. The time-travel API is flexible.

From a **developer UX** perspective, this is gold for debugging. If an agent failed or made a poor decision, you don't have to run everything again from scratch; jump to that point, adjust, and run the last steps differently. It's analogous to using a debugger in code to rewind to a breakpoint.

Implementation: Using the Checkpoint API

LangGraph's reference shows methods like `graph.get_state(thread_id)` (to get latest state), `graph.get_state_history(thread_id)` (to get all checkpoints for a thread), `graph.update_state(thread_id, checkpoint_id, new_state)` and of course `graph.invoke/stream` with checkpoint config to resume ⁵³ ⁵⁹. These give you programmatic control for building things like:

- A “resume conversation” feature in a chat app (just reuse `thread_id`).
- A UI where a user or dev can click a past step and edit it, then continue (like a visual debugger for the agent).
- Rolling back unwanted actions: e.g., if the agent was about to send an email via a tool and a human decides to stop it, you could roll back to before that tool and take a different path (maybe require human approval at that point next time – we'll see in HITL module how to integrate that).

Persistence Backends: The checkpoint approach means you can swap in different storage: - If you want lightning-fast in-memory only (for ephemeral runs), use `InMemorySaver`. - For production, a database is better. Postgres is one option ⁴⁸; Redis is another popular one ⁶⁰ ⁶¹. The Redis integration even supports vector storage for memory (tying into next module) and is optimized for speed and scaling ⁶² ⁶⁰. - There's also a `SqliteSaver` as seen in some tutorials (good for local prototypes, saving to a file).

Best Practices and Trade-offs

- **When to checkpoint:** By default, LangGraph checkpoints after every node execution. This is usually fine. If performance is a concern (writing to DB each step), you might adjust to checkpoint less frequently or use a faster store. But the overhead is generally minor compared to an LLM call. Having every step checkpointed is safest for maximum recoverability.
- **Thread ID management:** Ensure you generate unique thread IDs for independent sessions. You might use user IDs or random UUIDs. If you reuse a thread ID for unrelated interactions, states will collide. Also, cleaning up old thread data from a persistent store periodically might be needed (LangGraph likely has methods to delete old checkpoints if not needed [63](#) [64](#)).
- **State size:** The state can grow (especially if storing full conversation history). Checkpointing that repeatedly could become heavy. In Module 5, we'll discuss how to manage short-term memory (trimming or summarizing) to keep state size reasonable. Persisting only the necessary parts of state (e.g., you might not need to save large intermediate data if it can be recomputed) is a consideration.
- **Security of stored data:** If your state contains sensitive info, and you're saving to a DB or Redis, treat it as you would any persistent sensitive data (encrypt at rest if needed, secure the DB). The upside is you have a record of everything the agent did, which is great for audit; the downside is that record could include sensitive user queries or model outputs.
- **Crash recovery:** Design your system such that if your agent process restarts, it can fetch last checkpoints and resume if appropriate. For example, a background worker processing tasks with agents could look for incomplete threads after a crash and call `resume` on them. LangGraph's durable execution means the agent can pick up mid-conversation naturally [43](#) [65](#) .

Replay and Testing

Finally, checkpoint replay can be used for **automated testing** of agents: - You can simulate a conversation up to a point and then test various continuations programmatically. For example, test that if the agent's tool fails at step 3, and you resume with a fixed state, it responds with an apology. - You can also time-travel to measure the effect of prompts: run the agent normally vs. inject an extra knowledge in state at checkpoint and see difference in output – useful for evaluation.

LangGraph encourages this kind of iterative development. By making agents more like deterministic programs (with saved state and replay), we can apply software engineering techniques more rigorously to AI agents.

Suggested Exercise: Take an agent you've built (even a simple one) and simulate a failure scenario to practice checkpointing. For instance, run the agent with a checkpointer for a couple of steps, then intentionally stop it (you could just break out in code, or cause an exception). Then use the checkpoint data to resume the agent. Concretely: perhaps have an agent that plans a three-step task. Stop after step 2 (maybe simulate by throwing an exception in step 2 code). Then write a small script to load the last checkpoint and call `agent.invoke(None, config={...checkpoint_id...})` to complete step 3. This will help you verify that you can successfully save and resume agent state.

5. Memory Architectures (Short-Term, Episodic, Semantic)

Objectives: This module covers the different **memory architectures** for LangGraph agents, namely short-term memory (within a conversation or episode), long-term memory (persisting across sessions), and semantic memory (vector-based recall of knowledge). You will learn how LangGraph handles *short-term memory* through checkpoint persistence (as discussed) and how to extend agents with *long-term memory* stores for things like user profiles or facts over multiple sessions ⁶⁶ ⁶⁷. We'll also explore techniques for managing memory (trimming, summarizing) when conversations grow long ⁶³ ⁶⁴, and how to incorporate *semantic search* so agents can retrieve information by meaning (using embeddings) ⁶⁸ ⁶⁹. By the end, you should be able to implement an agent that remembers context over time and know how to prevent it from forgetting or overflowing its context window.

Short-Term Memory (Working Memory of the Agent)

Short-term memory refers to the agent's ability to remember what has happened so far in the current session or task. In a chat, this is the conversation history; in a multi-step workflow, it's the chain of reasoning and results so far. LangGraph provides this automatically via the **state** and **persistence layer**.

In LangGraph, enabling short-term memory is as simple as using a checkpointer to maintain the ongoing state for a given `thread_id` ⁷⁰ ⁷¹. For example, if you compile your agent with an `InMemorySaver` and use a consistent `thread_id` across calls, the agent will carry over the `messages` state from one `.invoke()` to the next:

```
# Short-term memory example
checkpointer = InMemorySaver()
agent = workflow.compile(checkpointer=checkpointer)

# User says hi (start a thread)
agent.invoke({"messages": [{"role": "user", "content": "Hi, I am Bob"}]},
            config={"configurable": {"thread_id": "conv1"}})
# Agent likely greets Bob...

# User asks a follow-up question in the same thread
response = agent.invoke({"messages": [{"role": "user", "content": "What's my name?"}]},
                        config={"configurable": {"thread_id": "conv1"}})
print(response["messages"][-1]["content"]) # should answer "Your name is Bob" or similar
```

Because we used the same `thread_id` (`conv1`), the agent's state from the first call (which presumably stored that user said "I am Bob") is persisted. On the second call, before processing "What's my name?", LangGraph loads the existing state for `thread1` – which includes the prior conversation. The LLM node sees that in `state["messages"]` and can answer correctly ⁵⁰ ⁷². If we had used different thread IDs, the agent would have no memory of Bob's name.

Short-term memory is essentially **thread-level persistence** ⁷⁰. It's great for multi-turn dialogues and multi-step workflows that span multiple API calls or user interactions. The persistence layer we set up earlier (InMemory, Postgres, etc.) takes care of storing and retrieving this context.

Episodic memory is a term often used similarly to short-term, implying memory of a specific episode (conversation session). In reinforcement learning literature, an “episode” ends and resets memory; in chatbots, an episode might be one conversation. With LangGraph, you decide when to treat something as a new episode (by using a new `thread_id` for a fresh conversation).

Managing short-term memory length: LLMs have a context window limit (e.g., GPT-4 might accept 8K or 32K tokens). If your conversation gets too long, you can’t keep sending all past messages to the model. LangGraph doesn’t automatically drop messages; it faithfully accumulates. Thus, it’s on the developer to trim or compress the conversation when needed.

Common strategies (LangGraph provides utilities for these [73](#) [74](#)):

- **Trimming messages:** Remove old messages once the list grows too large. You might keep only the last N messages or last N tokens worth. LangChain has a `trim_messages` utility that can drop messages from the start or using other strategies [75](#) [76](#). You can integrate this via a hook. For instance, in `create_react_agent` you can use `pre_model_hook` to trim the state’s messages before each LLM call [76](#) [77](#). This way, the state is full for persistence, but the model only sees a truncated history (maybe plus a summary, see below).
- **Summarizing:** Instead of dropping old context entirely, summarize it and keep the summary in state. LangChain supports making a summary of earlier messages (using an LLM itself). For example, after 50 messages, you could take the first 40, condense them into a summary message, and replace them with that in the state [73](#) [74](#). The agent then carries a shorter representation of long past context.
- **Episodic resets:** For some applications, you might decide that context older than a certain age is not relevant and start a fresh “episode.” For instance, a customer support agent might have separate sessions per issue.

LangGraph’s built-in memory guide lists these options: trimming, deleting specific messages, summarizing, and using checkpoints as a mechanism to page out history [73](#) [74](#). A snippet from docs: “Common solutions are: Trim messages, delete messages, summarize earlier messages, or manage checkpoints to store/retrieve history” [73](#).

In practice, to implement trimming in LangGraph, you can do:

```
from langchain_core.messages.utils import trim_messages,
count_tokens_approximately

def my_pre_model_hook(state):
    trimmed = trim_messages(
        state["messages"], max_tokens=1000, strategy="last",
        token_counter=count_tokens_approximately
    )
    return {"llm_input_messages": trimmed}
agent = create_react_agent(model, tools, pre_model_hook=my_pre_model_hook,
checkpointinter=InMemorySaver())
```

This hook ensures that just before the LLM is called, the messages are cut down to ~1000 tokens, keeping the last part of the conversation (strategy "last") ⁷⁶ ⁷⁷. The full list is still in the state if needed for other logic, but the LLM sees the trimmed version via `llm_input_messages`.

Long-Term Memory (Cross-Session Memory)

Long-term memory means the agent retains information beyond a single session or episode – knowledge that persists and can be recalled in future interactions, possibly days later. Examples: remembering a user's preferences, storing factual information learned, or maintaining a knowledge base of past events.

In LangGraph, long-term memory is facilitated by the **Store** interface (`BaseStore`). This is separate from the short-term checkpointing. The Store is essentially a key-value or key-document store where you can put and get data by keys and namespaces ⁷⁸ ⁷⁹.

To add long-term memory to an agent, you compile the graph with a `store` parameter (in addition to or instead of a checkpointer):

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()
agent = workflow.compile(store=store)
```

Now the agent has an associated store (like a database). The store can be in-memory for testing, or backed by Postgres (`PostgresStore`) or Redis (`RedisStore`) for persistence ⁸⁰ ⁸¹. The Store is intended for data that should outlive one thread and be queryable.

Usage of store: Within your node functions or tools, you can interact with the store. LangGraph's runtime can inject the store if you specify it in function signature (for instance, in a node function, use `*, store: BaseStore` and LangGraph will pass it) ⁸² ⁸³.

- To **write** to store: use `store.put(namespace, key, value)` ⁸⁴ ⁸⁵. The namespace is like a category (could be e.g. ("memories", user_id)). The key could be a unique ID or name for the memory, and value is typically a dict or other serializable data. For example, after a conversation, you might store `(("memories", "user123") -> {"data": "User likes sci-fi movies"})`.
- To **search/read** from store: `store.search(namespace, query="...")` returns items that match the query ⁸³ ⁸⁶. If an index (like embedding index) is set up, this can be semantic (more on that soon). Or you can retrieve by specific key with `store.get(namespace, key)` ⁸⁴.

Example: Imagine an agent that learns the user's name and stores it in long-term memory (so next session it knows them). We could do something like in the LLM node:

```
def call_model(state: MessagesState, config: RunnableConfig, *, store:
BaseStore):
    user_id = config["configurable"]["user_id"]
    # Retrieve any known info about user
    memories = store.search(("memories", user_id), query="") # get all
    memories for user
    info = "\n".join(item.value["data"] for item in memories)
```

```

system_msg = f"You are a helpful assistant. Known about user: {info}"
# If user asked to remember something:
last_user_msg = state["messages"][-1].content
if "remember" in last_user_msg.lower():
    store.put(("memories", user_id), str(uuid.uuid4()), {"data":
last_user_msg})
    response = llm.invoke([{"role": "system", "content": system_msg}] +
state["messages"])
    return {"messages": [response]}

```

This pseudo-code does a few things: - It pulls all stored memory for `user_id` and prepends it as context (system message) so the assistant is aware of user-specific info ⁸³ ⁸⁶. - If the user says something containing "remember", we store that message (or processed info) in the store under that user's namespace ⁸⁷ ⁸⁸. This effectively creates a durable memory. - Then it calls the LLM with both the known info and the conversation.

This pattern is shown in LangGraph docs for long-term memory ⁸³ ⁸⁹. By combining the store with short-term, the agent has both working memory (current conversation) and long-term knowledge (persisted facts).

Semantic Search in Memory: Often, long-term memory might contain many entries (imagine an AI that over time accumulates a lot of knowledge). Searching by keyword might not be effective. This is where **semantic memory** comes in – using vector embeddings to retrieve relevant information by meaning.

LangGraph's `Store` can be configured with an embedding index ⁶⁸ ⁶⁹. For example:

```

from langchain.embeddings import init_embeddings
embeddings = init_embeddings("openai:text-embedding-3-small")
store = InMemoryStore(index={"embed": embeddings, "dims": 1536})

```

Here we initialize an OpenAI embedding model, and tell the store to use it (with vector dimension 1536 for that model) ⁹⁰ ⁶⁹. Now, whenever we do `store.put(namespace, key, {"text": "some content"})`, the store will likely create an embedding for the text and store it. Then `store.search(namespace, query="I'm hungry")` will embed the query and return the stored items most similar in vector space ⁹¹ ⁹². This allows the agent to recall things that are semantically related, not just exact matches.

For instance, if the user's memory has "I love pizza" stored, and later the user says "I'm hungry", a semantic search might retrieve the "I love pizza" memory as relevant, which the agent can use to suggest pizza for lunch ⁹³ ⁹².

LangGraph's guide snippet demonstrates this: they store "I love pizza" and "I am a plumber", then search with "I'm hungry" gets the pizza memory ⁹¹ ⁹². This is powerful for personalization or any accumulated knowledge.

Implementing semantic memory: After setting up the store with an embedding index, usage is same `store.search`. Just ensure the content you put in store is under a consistent namespace (like

(user_id, "memories")) and includes the text field you want to search by. The `search` method can often take a `limit` parameter to return top N matches ⁹².

Memory Tools and Utilities

LangGraph also provides some **prebuilt memory tools** – these are tools the LLM can explicitly call to read/write memory. For example, a `RetrieveMemory` tool that the LLM could invoke like a database query, or `SaveMemory` tool. This is another design: rather than automatically pulling memories in the background, you let the LLM decide when to retrieve from long-term memory via tools. The guides mention "Prebuilt memory tools" ⁹⁴ as a concept. Depending on your design, you may either: - Always load some memories into context (good for small important facts like user name). - Provide a `search_memory` tool that the LLM can use if needed (good if there's a lot of data, to not overload context unless relevant).

Likewise, summarization of conversation can be automated by an `SummarizeHistory` tool that the LLM could call when it feels context is too long. These patterns can reduce developer-handling of memory and put more autonomy on the LLM (but that can be risky if the LLM doesn't realize it should call them).

Design Trade-offs in Memory

- **Storing too much vs forgetting:** Long-term memory is only useful if you retrieve the right pieces at the right time. Dumping an entire user history every time can bog down the model (and cost tokens). Semantic search helps pick the most relevant pieces. A recommended pattern is to retrieve a few top relevant memories for the current query and feed those in as context (like retrieval-augmented generation style).
- **Memory correctness:** The agent might retrieve an outdated or irrelevant memory. You might incorporate timestamps or validity checks in memory entries to avoid using stale info. Also consider that user may change preferences – the agent should update or invalidate old memory if contradictory new info comes (could implement a tool for the user to correct their data, or automatically prune contradictory memory).
- **Privacy and safety:** If an agent can remember anything, it could inadvertently surface sensitive info in the wrong context. E.g., remembering a user's secret and then blurting it out. Guardrails (Module 6) are important to ensure memory usage abides by policies (maybe labeling certain memories as private, or requiring user to explicitly ask for them).
- **Memory persistence backend:** For large-scale usage, a vector database (like Pinecone, Weaviate) might be used for semantic memory instead of the built-in store. The `BaseStore` in LangGraph could possibly integrate with such DBs through custom implementations.
- **Short-term vs long-term clarity:** A simple rule: use checkpoint (thread memory) for info that's only relevant in the conversation, and store (long-term) for info that should persist beyond it. There's overlap, but for example, the ongoing dialogue is short-term, while a user profile is long-term.

Example Scenario

To tie it together, consider an **AI personal assistant** agent: - It uses short-term memory to carry on a coherent multi-turn conversation. - It uses long-term memory to remember the user's name, birthday, favorite topics, etc., which it stores and retrieves from a `store` with `user_id`. - It uses semantic memory to remember articles or notes from past conversations; when the user asks a related question later, it can fetch those notes.

When the user comes back after a week, starting a new thread, the agent greets them by name because it loaded that from long-term memory. As the conversation goes on, the short-term memory accumulates. If the user's query hits on something discussed last month, the agent does a semantic search in stored memory and finds the relevant info to incorporate into its answer.

This is essentially giving the agent a **brain** with both short-term "RAM" and long-term "disk". LangGraph's architecture (checkpointer + store + embedding search) provides the pieces to build this.

Suggested Exercise: Augment a simple Q&A agent with long-term memory. For instance, create an agent that the first time you ask "What's my favorite color?" it says it doesn't know. If the user then says "My favorite color is blue. Remember that.", the agent stores that info. On a follow-up "What's my favorite color?", it should retrieve and answer "Blue." Implement this using a `PostgresStore` or `InMemoryStore` and test it across multiple `.invoke` calls (simulating separate sessions). This exercise will reinforce how to use `store.put` and `store.search` within your agent logic.

6. Human-in-the-Loop (HITL) & Guardrails

Objectives: In this module, we discuss strategies to keep a human in the loop and enforce guardrails on your LangGraph agents. You will learn about LangGraph's features for **pausing execution (interrupts)** to allow human review or input at critical points ⁹⁵ ⁹⁶. We'll examine typical HITL patterns like requiring approval for certain actions, letting humans correct the agent's state, reviewing tool outputs, etc. ⁹⁷ ⁹⁸. Additionally, we'll cover **guardrails** – automated checks or constraints to prevent undesirable outputs or actions, such as content moderation or limiting tools. By the end, you should know how to design agents that collaborate with humans for safety and reliability, and implement breaks in autonomy where needed.

Why Human-in-the-Loop?

While autonomous agents are powerful, unchecked autonomy can lead to errors or unwanted behavior. Human-in-the-loop means a person can **observe, intervene, and guide** the agent's decisions at runtime ⁹⁵. This is especially important in high-stakes applications (e.g., medical advice, financial decisions) or when an agent can perform actions (like sending emails, making purchases).

HITL can serve several purposes: - **Safety:** Prevent the agent from executing a potentially harmful or irreversible action without approval (e.g., deleting data, making a financial transaction). - **Quality control:** Ensure the agent's output meets certain standards. A human might review a draft email the agent wrote before it's sent. - **Error correction:** If the agent is on the wrong track (misunderstood user or context), a human can step in, correct the state or give a hint, then let it continue.

LangGraph is built to accommodate HITL easily because of its persistent state and checkpointing. The idea is to **pause** the agent at a certain point, allow human inspection/modification, then **resume**.

Using Interrupts to Pause Execution

LangGraph introduces the notion of **interrupts** – essentially breakpoints in the graph execution where it should halt until told to continue ⁹⁶. There are two types: - **Static interrupts**: set in advance at specific nodes or edges. For example, you could declare that the graph should always pause *before* executing a particular tool node (say, an API call to charge a credit card) ⁹⁹. - **Dynamic interrupts**: triggered conditionally during execution. For instance, inside a node function, you might decide to interrupt if some condition in state is met (like the agent's score of confidence is low, or it produced content needing moderation) ⁹⁹.

LangGraph API allows specifying `interrupt_before(node)` or `interrupt_after(node)` when building the graph, or calling an `interrupt()` function within a node to signal a stop ⁹⁶ ⁹⁹. When an interrupt is hit, the agent saves a checkpoint and then yields control – effectively doing nothing more until explicitly resumed by a human (or code).

Because of persistence, the agent can remain paused indefinitely ⁴⁴. The human can come minutes or hours later to resume, and the state is intact ⁴⁴ ⁴⁵.

Example: Suppose we want human approval before an agent uses the `delete_user_account` tool. We could do:

```
workflow.add_node("delete_account", ToolNode([delete_user_account]))
workflow.add_edge("LLM", "delete_account")
workflow.interrupt_before("delete_account") # pause here for approval
```

Now, whenever the agent is about to execute that node, it will halt. The system (your code) could then alert an admin or present a prompt: "Agent wants to delete user account X. Approve?" The human can inspect the state (which should include which account, etc.), perhaps modify something or just okay it.

To resume, the human triggers the continuation, which might be something like:

```
agent.invoke(None, config={"configurable": {"thread_id": current_thread,
"checkpoint_id": checkpoint_id}})
```

This resumes from the paused checkpoint (as we learned in module 4). Alternatively, LangGraph might have a helper to resume an interrupted thread easily.

While paused, a human could also **edit the state**. Maybe the agent was deleting account "mike@gmail.com" but the human knows that's wrong – they could change that in state or even change the next node (some advanced usage allows altering the command for goto).

HITL Design Patterns

The LangGraph documentation enumerates typical patterns implementable with interrupts and Commands ⁹⁷ ⁹⁸:

- **Approve or reject:** As described, pause at a critical step for approval. If approved, continue normal flow; if rejected, perhaps skip that step or route to an alternate path (e.g., ask the user

for different input). You can implement the alternate path by having the human's input (approve/reject) update the state or set a flag that the agent logic checks on resume and changes course

100 .

- **Edit graph state:** Pause, let human edit the state, then continue 101 102 . For example, an agent wrote a summary but missed a key point – a human can insert that point into the state (maybe into the messages or a separate state field) and then resume the agent to finalize output with the correction.
- **Review tool calls:** Pause right after the LLM decides a tool call but before executing it 98 . This allows a human to see "Agent is about to execute: `send_email(to=XYZ, content='Hello')`" and either approve, modify the parameters, or cancel. This pattern is effectively intercepting the tool usage. You could implement it by interrupting *before* the ToolNode as above, or inside the LLM node if it produces a tool call that matches certain criteria (maybe sensitive action), then requiring manual validation.
- **Validate human input:** Sometimes the human user's input might need validation (like confirming if they truly want to perform some sensitive request). This can be seen as a form of guardrail – e.g., if user says "delete all my data", agent might interrupt to ask a human operator to verify the user's identity or intention before proceeding.

Implementing these patterns might involve using the `Command` object in LangGraph. The `Command` is used in node functions to explicitly route to a certain node. For HITL, you might have a node that returns `Command(goto="pause")` to effectively yield to a waiting state. However, simpler is using interrupts which automatically handle it by persistence.

LangGraph ensures that when an interrupt happens, the state is saved so you can later resume exactly from there 44 45 .

Guardrails and Moderation

Beyond involving a human, you also want some automated **guardrails** to keep the agent behavior in check: - **Content moderation:** Ensuring the agent doesn't output offensive or disallowed content. You can integrate an automated content filter (like OpenAI's moderation API or a keyword filter) in the workflow. For example, after the LLM produces a message, you could have a step that scans it. If it flags something, you either modify it or trigger an interrupt for human review.

LangGraph could incorporate this by: - Having a post-processing node: LLM -> ModerationCheck -> either proceed or go to a safe output. The ModerationCheck node could examine `state["messages"][-1]`, and if bad, perhaps replace it or set a flag and route to an alternate path (maybe the agent says "I cannot comply with that request"). - Or using a `pre_model_hook` to filter user input and refuse if it contains disallowed content.

- **Tool usage limits:** Imposing limits like the agent can't call tools more than N times to avoid runaway loops (also a cost control). This can be enforced by adding a counter in state and a check: if counter > N, route to END with some apology or ask human for whether to continue.
- **Time or cost limits:** Similar concept, you might measure how long or how many tokens have been used (LangChain callback can track tokens 103 104). If exceeding budget, interrupt or stop.

- **Restricting actions by policy:** For instance, a guardrail that the agent cannot provide medical advice beyond a certain level – you could implement a check on the conversation content and if it enters a “danger zone” topic, require a human doctor to review the response or simply respond with a safe completion.

LangChain has an initiative called Guardrails (by Shreya Rajpal) which can validate LLM outputs against a schema or policy. One could integrate that in a LangGraph node as well, to systematically sanitize outputs.

Human feedback: Another angle of HITL is continuous improvement – using human ratings to refine the agent (RLHF etc.), which is beyond immediate scope but good to keep in mind.

Implementation Tips for HITL

- Make use of the **LangSmith** or logging to present info to humans. If you have a UI, you might display the agent’s chain-of-thought to the reviewer so they know why it’s asking for approval.
- Keep the human interface simple: when pausing, output a clear message like “Agent paused before executing tool X. Awaiting instruction: [Approve]/[Edit]/[Abort].”
- After human intervention, mark it in the state (for audit). E.g., add a message “(Action approved by human)” so there’s a record in the state for future reference or analysis.
- Be mindful of not overly interrupting. Decide which guardrails can be automated vs which truly need a person. Too many pauses can make the system inefficient.

Real-World Example

Think of a **clinical decision support agent** for doctors. It can suggest diagnoses or treatments, but you absolutely want a human doctor to approve any final recommendation. You could design the agent to do all the reasoning and then pause when it has a recommendation. The doctor reviews, maybe adjusts a dosage, and then the agent can finalize the report or order with those adjustments. Also, content guardrails would ensure the agent doesn’t give advice outside certain bounds.

Another example: a **customer support chatbot** that handles refunds. For small refunds it auto-approves via a tool call to payment system; for large refunds, it pauses and asks a human manager to approve. This hybrid approach saves time on small issues but retains human control on big ones.

LangGraph, by letting you slot in these checks and stops, helps you build such nuanced flows without tearing down the whole architecture. You just weave in interrupts and custom logic at needed points.

Suggested Exercise: Implement a simple guardrailed agent. For instance, an agent that can tell jokes but should not use any profanity. Integrate a profanity filter: after the LLM crafts a joke, scan it (you can use a simple list of bad words to check). If profanity is found, either have the agent try again (loop back to LLM with instruction to avoid that language) or have it output a sanitized version. Test the agent with a prompt that might lead to a bad word to ensure your guardrail catches it. This exercise combines conditional logic, possibly a loop, and demonstrates an automated safety check.

7. Streaming, Observability & Developer UX

Objectives: In this module, we cover features that improve the runtime experience and debuggability of LangGraph agents. You'll learn how to enable **streaming** token-by-token outputs for better user experience ¹⁰⁵ ¹⁰⁶, and how LangGraph supports streaming of intermediate steps as well. We will discuss **observability** tools – using LangSmith tracing to visualize agent runs ¹⁰⁷, logging, and debugging techniques that LangGraph enables (like introspecting state at checkpoints). Additionally, we'll mention aspects of developer experience: the ability to visualize the graph structure, easily test parts of the workflow, and use interactive notebooks or UIs to iterate quickly. By the end, you should be able to configure your agent to stream responses to users and leverage tracing to understand its behavior under the hood.

Streaming Outputs

Streaming refers to outputting the LLM's response gradually (token by token or chunk by chunk) rather than waiting for the full completion. This leads to a more interactive feel – the user sees the answer appear as it's being generated, which is crucial for UX in chat interfaces.

LangGraph has first-class support for streaming at both the LLM level and the graph level ¹⁰⁸ ¹⁰⁶. You likely noticed in earlier examples we used `agent.stream()` with `stream_mode="values"` or `"messages"`.

How it works: - If the underlying LLM supports streaming (e.g., OpenAI ChatCompletion with `stream=True`), LangChain's model can yield tokens or partial messages. - LangGraph's `stream` method will then propagate those partial results through the graph.

For instance, `for chunk in agent.stream(input, stream_mode="values"):` yields a series of states (or outputs) as the agent executes step by step ¹⁰⁹ ¹¹⁰. Typically: - You get intermediate messages as they're produced by the LLM node. If the LLM is streaming its answer, each token can be yielded in a chunk. - If an agent has tool calls, you might see nothing during the tool (since that's instantaneous from code), but then stream the next LLM output.

In practice, to implement streaming to a front-end, you might call `agent.stream` and for each chunk, extract the latest message content and send it to the UI. The final chunk will indicate the completion.

Token-by-token reasoning display: LangGraph can even stream the agent's reasoning process, not just final answer. Because each intermediate node's completion is checkpointed, you could output things like "(Agent deciding which tool to use...)" if you want. Many developer UIs show the thought process live (though to end users, you might hide that).

From LangChain's perspective, streaming in LangGraph is similar to normal streaming but orchestrated along the graph.

Observability with LangSmith and Tracing

Building and debugging complex agents can be challenging. **LangSmith** is LangChain's observability platform that integrates nicely with LangGraph ¹⁰⁷. It allows you to visualize the execution trace: every LLM call, tool call, node transition, etc., as a sequence (or graph).

By enabling tracing in LangGraph (often just by setting an environment var or using a context manager), every run is recorded. You can then use a dashboard to see: - The sequence of calls (graph runs) ¹⁰⁷ ¹¹¹ . - Timing info, token counts, and even cost if integrated ¹¹² . - The input/output at each step, which is fantastic for debugging logic issues.

To enable:

```
import os
os.environ["LANGCHAIN_TRACING"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "MyLangGraphApp"
```

By doing this (and having a LangSmith API key configured), each `agent.invoke` or `agent.stream` call will log a trace. The docs mention using `enable_tracing_for_your_application` ¹¹³ ¹¹⁴ - which likely corresponds to environment or using `with LangChainTracer()`.

Once traces are collected, you can go to the LangSmith UI and see *Graph runs*. These might depict nodes and edges as executed. LangSmith is aware of LangGraph, so it can show a “graph run” comprising multiple sub-runs (LLM calls are sub-runs, etc.) ¹⁰⁷ ¹¹¹ .

What does this give you? - **Debugging:** If an agent gave a wrong answer, you can inspect what it was thinking. Maybe it did a search but picked a wrong snippet. By seeing the trace, you find the issue and can adjust the prompt or logic. - **Performance tuning:** The trace includes token usage and time per step. You might notice one tool call is slow or an LLM call wasted many tokens thinking in circles. That insight can inform optimizations (like adding constraints or using a smaller model for some steps). - **Error analysis:** If an error occurred, the trace shows where. E.g., a tool threw exception – you see that and can add handling.

Additionally, LangSmith allows attaching feedback or evaluations. For example, you could label a run as good or bad, or have test criteria that automatically evaluate the final answer (module 9 will cover evals). Observability is the foundation for systematically improving your agent.

LangGraph’s **tracing conceptual guide** provides more details, but key point: use tracing early in development. It’s much easier to develop when you can literally see what the agent did on each turn.

Beyond LangSmith, simple logging is valuable. You can always print or log the state at certain points if you run in a console. But with many nested calls, structured tracing is better.

Developer Experience Enhancements

A few things make LangGraph nicer to work with as a developer:

- **Graph visualization:** Because your workflow is a graph, it’s natural to want to visualize it. LangGraph’s `agent.get_graph().draw_mermaid()` (as seen commented in code earlier ¹¹⁵) suggests you can generate a diagram (Mermaid or image) of the graph structure. This helps ensure the flow is as you intended (especially for complex branching). If the library is installed with graphviz, you might directly plot it in a Jupyter notebook. Visualizing the static graph is a quick sanity check.

- **Modular design:** You can test individual nodes in isolation. For example, test your tool functions separately (outside the agent) to ensure they behave. Test your LLM prompts separately by calling the model on expected inputs. This unit-test-like approach speeds up development rather than always running the full agent.
- **Interactive development:** Jupyter notebooks or similar environments pair well with LangGraph. You can iterate on the agent, run it on sample inputs, inspect intermediate results (since you can peek into state or partial outputs). The fact that you can `invoke` and get the whole state back as a Python dict means you can introspect it easily in code.
- **Hot-reloading and updates:** If using LangGraph in a live system, consider building ways to update prompts or logic without restarting everything. Because the logic is in Python, you may redeploy code to change a node's function. But perhaps design your prompts and tool configurations to be data-driven (maybe stored in a config) so you can tweak without full code changes.
- **LangGraph Studio (Prototype):** The marketing material references a visual prototyping tool called LangGraph Studio ¹¹⁶ ¹¹⁷. This likely allows drag-and-drop building and one-click deployment. If available, it can significantly boost DX (developer experience). While that might be part of LangChain's platform, not open source, it indicates the direction: a GUI to design and test the agent flows.
- **Testing harness:** It's good practice to write automated tests for your agent. This might involve simulating certain user inputs and checking the agent's response or checking that it called the right tool. LangGraph's determinism (given same random seed, LLM calls can still vary a bit if they have randomness, but you might set `temperature=0` for test scenarios) allows writing such tests.
- **Observability for users:** Consider showing the end-user some of the agent's reasoning or actions if appropriate (e.g., "Searching database..." messages). This can build trust that the agent is doing something sensible. Streaming intermediate steps in a user-friendly way (like showing typing indicator or "Agent is looking up info") can improve UX.

Real-time Token Streaming Example

If using `stream_mode="messages"`, LangGraph can yield individual message objects. You could print partial content:

```
for message, metadata in agent.stream(input, stream_mode="messages"):
    if message.role == "ai":
        # This is a piece of the AI's output message
        print(message.content, end="", flush=True)
```

This way you print tokens as they come (ensuring not to newline until done). If you've seen ChatGPT type out answers, that's the effect.

LangGraph's streaming also covers **intermediate reasoning streaming**. Some applications show each thought and action as it happens (like an AI researcher agent that prints "Thought: I should search for

X\nAction: search_web"). For a developer or power user mode, you can do that by tapping into the `messages` stream including tool messages. You might filter and display those in an debug console.

Cost and Performance Monitoring

As part of observability, tracking cost is key (since LLM calls cost money). LangSmith can track token counts and cost if you provide pricing info for the model ¹¹². Or you can manually use callback handlers (like `OpenAICallbackHandler` as in the GitHub discussion) to accumulate token usage ¹⁰³ ¹⁰⁴. Logging how many tokens each conversation uses will help with cost control (discussed in module 9) and optimizations.

Developer Iteration Cycle

LangGraph encourages a more **structured approach** to building agents. Instead of prompt engineering in one big loop, you break the problem into nodes and test each. The observability tools then let you verify the integration. This can dramatically shorten the iteration cycle because you catch issues at the graph design phase (e.g., missing an edge, or a wrong condition) rather than scratching your head on why a giant prompt didn't work.

To summarize: - Use streaming to improve responsiveness of your app (users get quicker feedback). - Use tracing to improve your understanding and debugging of the agent. - Utilize visualization and logs to refine the design. - All these make building with LangGraph a smoother experience relative to treating the LLM as a black box.

Suggested Exercise: Enable tracing on a LangGraph agent and purposely introduce a bug (for example, have a conditional edge always go to a wrong node). Run the agent with tracing and use the trace visualization to identify the bug. Then fix it and verify the trace shows the correct flow. This will give you hands-on experience with the tracing tools and an appreciation for how they catch logic errors in the graph.

8. Multi-Agent & Orchestration Patterns (Advanced)

Objectives: This optional advanced module explores patterns for building systems with **multiple agents** in LangGraph. We'll cover orchestrations such as agents cooperating under a supervisor, hierarchical agents, and explicit multi-agent workflows ¹¹⁸ ¹¹⁹. You'll learn how to have agents call each other (or treat one agent as a tool of another), how to coordinate their turns, and when multi-agent setups are beneficial. We will also discuss the design considerations of multi-agent systems (like avoiding infinite loops between agents, ensuring clear communication protocols). By the end, you should grasp several multi-agent architectures and how to implement them using LangGraph's primitives (with examples for a supervisor-agent pattern and a hierarchical team of agents).

When to Use Multiple Agents?

Single-agent systems can handle many tasks, but there are scenarios where splitting into multiple specialized agents is useful: - **Specialization:** One agent might be an expert coder, another a tester. They can work together, each doing what it's best at. - **Parallelism:** Multiple agents can work on different subtasks concurrently (though LangGraph execution by default is sequential, you can conceptually parallelize by running subgraphs). - **Dialogue or Debate:** Agents can engage in a conversation (like playing roles: proponent and critic) to reach a better solution. - **Orchestration logic:**

A top-level agent (or non-LLM controller) can manage sub-agents. This helps structure very complex workflows.

LangGraph is well-suited for multi-agent because a graph naturally can include multiple nodes that are themselves LLM calls (possibly with different prompts or even models) ¹⁹. Each such node can be considered a distinct agent if it has its own identity or role.

Pattern 1: Supervisor-Agent (Tool-Calling) Pattern

One straightforward architecture: a **supervisor agent** that delegates tasks to other agents treated as tools ¹¹⁸ ¹²⁰. In LangGraph: - You have a main agent (LLM) that doesn't do all the work itself but rather decides which sub-agent (tool) to invoke. - Each sub-agent is implemented as a function (or a smaller LangGraph) and registered as a tool for the supervisor.

Concretely, imagine Agent A (supervisor) has tools: `Agent_B_tool`, `Agent_C_tool`, corresponding to two other agents (B and C). When A's LLM output says "use tool `Agent_B_tool` with input X", LangGraph will call Agent B to handle X, then return the result to A.

Using `InjectedState` in LangGraph, you can even pass parts of the supervisor's state into the sub-agent call ¹²¹ ¹²². The code snippet in docs shows an example where `agent_1` and `agent_2` are defined as Python functions that take `state: Annotated[dict, InjectedState]` - meaning they can access the supervisor's state directly ¹²¹ ¹²². They return a string (tool output). The supervisor is built via `create_react_agent` with these agents as tools ¹²³.

This pattern effectively treats sub-agents as **black-box skills**. The supervisor LLM handles the reasoning of *when* to use which skill.

Use case: A research assistant agent might have a coder sub-agent for writing code and a writer sub-agent for writing prose. The supervisor reads the user's complex request, and decides "this part I'll send to coder agent, that part to writer agent, then combine results."

Pattern 2: Multi-agent with Explicit Turn-Taking (Supervisor without Tools)

Another approach is to orchestrate multiple agents by explicitly controlling their dialogue. For example, **chain-of-thought debate**: Agent 1 proposes an idea, Agent 2 criticizes, Agent 1 responds, etc., until termination.

You can implement this with a graph where nodes are the agents and edges control the turn order: - Node Agent1: LLM prompt for agent 1's turn. - Node Agent2: LLM prompt for agent 2's response. - Edges: Agent1 -> Agent2 -> Agent1, forming a loop until some condition.

This is similar to a chat simulation between two AIs. You might include a termination check (e.g., if an agent says "I rest my case" then go to END).

LangGraph can coordinate this by housing a shared state of the conversation and alternating nodes. The *supervisor* in this case is just the static graph that alternates turns, rather than another LLM.

Docs hint at a *custom multi-agent workflow* where agents are just nodes and edges define the sequence explicitly ¹²⁴. They mention explicit control via normal edges vs letting them decide via commands.

Pattern 3: Hierarchical Agents

For very complex tasks, you might organize agents in a hierarchy of supervisors ¹²⁵ ¹²⁶ . For example: - A top-level agent breaks a project into two subtasks and delegates to two *team* agents. - Each team agent is itself a supervisor of a group of sub-agents.

LangGraph allows you to even put entire compiled graphs as nodes in a higher-level graph (treating a subgraph as a node) ¹²⁷ . The snippet in docs shows adding `team_1_graph` and `team_2_graph` as nodes in a top-level graph ¹²⁷ ¹²⁸ . Those `team_x_graph` were themselves StateGraphs compiled earlier ¹²⁹ ¹³⁰ .

So you can have a **nested graph** structure: top graph has nodes that are actually entire agent workflows. This works because each compiled graph (sub-agent) can be invoked like a function; LangGraph likely wraps it so that from top-level's perspective, it's just another callable that takes state and returns state.

In the code: - Team 1 supervisor (LLM) decides which of team 1's agents to call, etc. - Top supervisor decides which team to call ²⁶ ¹³¹ .

This hierarchical approach scales to complex multi-step processes where dividing responsibilities yields clarity.

Example: Building an essay: - Top-level agent decides to either do research or writing at a given time. - It has two sub-agents teams: Research Team and Writing Team. - Research Team may have agents for web search and fact-checking. - Writing Team may have agents for outline and drafting. - The top-level orchestrates: first call Research Team (which internally might call search agent multiple times), then call Writing Team with gathered info.

It's sophisticated but LangGraph's design encourages breaking into such subgraphs for maintainability.

Considerations in Multi-Agent Systems

- **State sharing vs isolation:** Do agents share the same memory or have separate memory? In LangGraph, if agents are just nodes on one state, they share the state (messages etc.). If you want them to have private memory, you might need to structure state to have separate subfields for each agent, or run them in separate threads that occasionally sync. The patterns above mostly share state so agents hear each other.
- **Communication protocol:** If agents are speaking to each other, ensure their prompts are set such that they know what to do with the other's messages. Often you'll give each a role (e.g., one is a questioner, one is a solver). Clear instructions are needed to avoid them converging to the same style or getting confused.
- **Avoid infinite loops:** Multi-agent loops can degenerate if not properly terminated. Set conditions for stopping, like a max number of turns or a convergence criteria, to avoid them chatting forever.
- **Cost:** More agents = more LLM calls, possibly more cost. Ensure the benefit outweighs cost. Perhaps use smaller/cheaper models for sub-agents if possible.

- **Debugging complexity:** Tracing becomes even more important here, to see which agent said what. LangSmith traces will show nested runs (sub-agent runs inside the overall run). Use that to debug miscommunications.
- **Use of commands:** When an agent's output determines the next step, you can parse it manually as we did with toolcalls, or you can design the agent to output a structured command (some frameworks have used XML or JSON outputs to indicate which agent to call next). LangGraph's `Command` type is one way to formalize that ¹³² ¹³³. For instance, an agent's LLM could be prompted to always output a JSON with field "next_agent": either "team_1_agent_1" or "end", which is then parsed and used in `Command(goto=...)` ¹³² ¹³³. This requires careful prompt engineering but yields precise control as shown in the hierarchical example (they parse `response["next_agent"]`).

Real-World Use Cases

- **Collaborative assistants:** e.g., one agent is good at code, another at math, another at explanation. Together they solve a problem the single model might fail at. This concept is akin to "society of minds" approach.
- **Simulations:** Multi-agent environments (like generative agents simulating a small town, where each agent is a character). LangGraph can simulate time steps or events by orchestrating interactions between agents systematically.
- **crewAI pattern:** The mention of crewAI in IBM site suggests an orchestrator agent calling multiple specialized actors (like crew members). This is exactly the supervisor-with-tools pattern.

By structuring multi-agent interactions in LangGraph, you avoid a lot of manual prompt juggling. The graph does the coordination, and each agent runs when it's supposed to.

Suggested Exercise: Create a simple multi-agent setup in LangGraph: a "Questioner" agent and an "Answerer" agent. The Questioner's job is to ask for clarifications, and the Answerer tries to answer. The user asks a vague question to the system; the Questioner agent should first ask a clarifying question; the user (or a simulated user) responds; then the Answerer agent gives the final answer. Implement this sequence with at least two LangGraph LLM nodes (one with a prompt to act as clarifier, one as answerer). Ensure the conversation flows correctly via the graph edges. This will give you practice in orchestrating two LLMs with distinct roles.

9. Evaluation, Testing, Safety & Cost Control

Objectives: In this module, we focus on practices for evaluating and testing your agent, ensuring safety, and controlling costs. You will learn how to systematically **evaluate** agent performance using both automated tests and human feedback, possibly leveraging LangSmith's evaluation features or custom metrics. We'll discuss testing strategies (unit tests for agent logic, integration tests for end-to-end behavior). On the safety front, we revisit content moderation and bias, and how to use guardrails and iterative prompting to minimize harmful outputs ¹³⁴. Finally, we address **cost control**: monitoring token usage, setting budgets, selecting model variants for cost vs quality trade-offs, and implementing limits on tool calls or steps. By the end, you should have a toolkit for maintaining quality, safety, and efficiency of your LangGraph agent in production.

Evaluation and Testing of Agents

Unlike traditional software, LLM agents can have nondeterministic behavior and subtly wrong outputs. Thus evaluation is both about correctness on tasks and about adherence to desired behavior.

Automated evaluation:

- **Unit tests for logic:** As mentioned, test each piece of the graph. If you have a function node that decides something, write a direct test for it with a fake state input. If you have prompts, you can even test them with a stub LLM or a known completion.
- **End-to-end tests:** Prepare sample conversations or tasks with expected outcomes. Run the agent (maybe with temperature 0 for consistency) and check the outcome. For example, if building a calculator agent, test that “What is 2+2?” yields “4”. Use the `agent.invoke` and parse result to compare with expected. LangSmith’s **evaluation** module can help here – you can programmatically compare outputs and mark pass/fail.
- **Regression testing:** Keep a log of known issues and fix them, then add tests to ensure they don’t recur. Because LLM output can vary, consider using fuzzy matching or semantic similarity to evaluate output rather than exact string compare. LangSmith supports embedding-based evals (to see if answer is semantically similar to a reference).

Human evaluation:

- Use a rating system in production where end-users can give thumbs up/down. These labeled traces can be analyzed to identify failure modes.
- For critical tasks, do a manual review of a sample of agent outputs periodically.

LangGraph integrates with LangSmith to record runs. You can leverage that by adding **Expectations** or custom evaluation functions on traces. For instance, verify that if the agent used a tool, the final answer includes info from that tool (some consistency check).

Measuring performance: If it’s a chatbot, maybe you want to measure average turns to resolve an issue. Or if it’s a writing agent, measure grammatical errors (with another model or tool). Define metrics relevant to your domain.

One can also simulate users (with another agent or script) to test how the agent handles various inputs, including edge cases and adversarial queries (like prompt injections or toxic input). This is part of safety testing.

Safety Considerations

We touched on guardrails in Module 6. To reinforce:

- **Content moderation:** Always run user inputs and possibly model outputs through a filter if your domain demands it. OpenAI’s API offers a moderation endpoint you can use as a tool or pre-check. LangGraph can incorporate that as an initial step or final step. E.g., before the model responds to user, check if response content is flagged. If so, either refuse or heavily modify it to be safe.
- **Prompt injection defense:** If your agent uses tools or follows certain instructions, a user might try to inject something like “Ignore previous instructions and...”. Mitigate by strict prompting (don’t let user content directly influence system role instructions or chain logic). Perhaps do not store user messages in the same place as your system messages, or always append system rules *after* user message in the final prompt to reinforce them.
- **Tool misuse:** Ensure the agent cannot use tools in ways you didn’t intend (like using a code execution tool to do harm). Keep dangerous tools out or behind an approval step. Possibly constrain tool inputs (e.g., if providing a shell tool, sandbox it strongly).
- **Privacy:** If storing long-term memory about users, have policies on data retention and obtaining user consent. Provide ways to wipe a user’s memory if requested (you’d delete from the store or not load it if user opted out).
- **Bias and fairness:** Evaluate whether the agent’s outputs show bias or inappropriate assumptions, especially if the agent interacts differently with different user groups. This is hard as it’s in the model pretraining mostly, but you can mitigate by

instructions and by filtering outputs. - **Hallucinations:** While not exactly a moral safety issue, giving wrong info can be critical (like wrong medical advice). Methods to reduce hallucinations: retrieval augmentation (ensuring the agent cites sources from memory), asking the agent to double-check via an alternate tool (like cross verifying facts via search), or final validation by a rule-based check if possible. For instance, for a math agent, you can implement a secondary calculation to verify the result.

LangGraph's structure helps because you can insert such validation nodes.

Continuous improvement: Use real user interactions (if you can log them ethically) to find where safety filters triggered or should have triggered. Update your prompts or guardrail rules accordingly.

Cost Control

Running LLM agents, especially with large models, can incur significant costs. Some strategies: - **Model selection:** Use the smallest, cheapest model that achieves acceptable performance for each part. Maybe a GPT-4 for complex reasoning, but GPT-3.5 for simple responses or a domain-specific smaller model for specialized tasks. LangGraph can mix models – different nodes could call different endpoints/providers ¹³⁵ ¹³⁶. - **Limit loops:** Put a cap on how many cycles the agent can do. For example, if after 5 tool uses it hasn't finished, maybe stop or escalate to a human. This prevents runaway token usage on a query that confuses the agent. - **Caching:** If identical questions repeat, you could cache answers. LangChain has caching support (e.g., in-memory or Redis cache for LLM responses). If applicable, integrate that to avoid calling the model for frequent queries. You might even precompute certain tool results if they're known (like a knowledge base). - **Monitoring token usage:** Use callbacks or LangSmith to record tokens per conversation ¹¹². Analyze which parts use the most. Perhaps an intermediate step uses a lot of tokens for little benefit – can you shorten prompts or cut it out? - **Budget enforcement:** If you have an allowance per user or per session, track it. You can maintain a token counter in state. Each time an LLM node runs, increment by tokens used (you can estimate with token counting utils or actual usage if available). If budget is about to exceed, either alert the user or ask if they want to continue (maybe they'll accept potential cost). At worst, stop the agent politely: "I'm sorry, this session has reached the usage limit." - **Batch processing where possible:** If your agent can handle multiple tasks in one prompt, sometimes that's cheaper than multiple separate prompts (though might increase complexity). This is more task-specific.

- **Tune prompts to be concise:** Long system or user prompts cost tokens. Trim needless verbiage in instructions. Use memory and context wisely; don't feed entire history if not needed (the trimming strategies help here, also semantic search to only inject relevant bits).
- **Use vector store instead of raw text memory:** If your long-term memory is large, querying it via vectors and only injecting a summary or top facts uses far fewer tokens than dumping pages of text into the prompt.

LangSmith or other observability can give an overall cost per run. You can set up alarms if cost per run spikes.

The GitHub discussion we saw had someone asking how to enforce cost threshold ¹³⁷. The solution was to capture tokens used and presumably make decisions. So you might do:

```
if callback_handler.total_cost > MAX_COST:  
    # maybe interrupt or cut off certain expensive steps
```

Keep in mind tokens = cost roughly, so controlling tokens controls cost.

Example: If using OpenAI, GPT-4 is ~30x cost of GPT-3.5 per token. Maybe use GPT-4 only when necessary. An approach: run a cheap model to get a draft or do quick checks, and only if needed, call the big model. LangGraph could incorporate that by having a node that tries a cheap path first.

Bringing It All Together: Deployment Readiness

Testing, safety, cost – all feed into whether your agent is ready for production: - Test it thoroughly, both in happy paths and adversarial cases. - Put guardrails and have monitoring for any unsafe outputs (maybe log them and have a human review periodically if something slips). - Keep an eye on usage patterns and optimize for cost if usage scales.

Often, deploying an agent might start with a **pilot**: limited audience or limited functionality, gather real data, then iterate.

For evaluation, consider key metrics: - Accuracy or success rate on tasks (if definable). - User satisfaction (via ratings). - Safety incidents (number of times content got blocked or needed human intervention – ideally zero). - Cost per conversation or per resolved task.

Use those to drive improvements in prompts or logic.

Suggested Exercise: Set up a simple evaluation scenario for your agent: define 5 input queries and the ideal responses. Write a script to run your agent on those and measure either exact match or semantic similarity to the ideal. For any that fail, tweak either the agent's logic or prompt and test again. This will simulate a mini evaluation loop. Additionally, try intentionally giving the agent a prompt that should be refused (like "Tell me how to do something illegal") and verify that your guardrail indeed refuses/redirects. If it doesn't, strengthen your guardrail and test again. This ensures you evaluate both normal performance and safety behavior.

10. Production Deployments & Capstone Use Cases

Objectives: In this final module, we discuss how to deploy LangGraph agents in real-world applications and highlight capstone use cases that bring together all concepts. You'll learn considerations for **production deployment**: choosing the right infrastructure (serverless vs persistent servers), scaling to many users, ensuring reliability (using persistent memory and durable execution features we covered ¹³⁸ ¹³⁹), and monitoring in production (using logs/traces to catch issues). We will also walk through a couple of **capstone scenarios** – fully realized agent systems – and analyze how they utilize the modules we've covered. These might include an autonomous customer support agent, a research assistant, or a multi-agent workflow for a complex task. By the end, you should have a clear picture of how to assemble LangGraph's components into a robust deployed solution.

Preparing for Deployment

Before deploying, ensure: - All keys and configuration (API keys for LLMs, tools, database URIs for stores) are securely stored (not hard-coded, use environment variables or a secure vault). - The dependencies (langgraph and others) are installed on your server environment, and you've pinned versions if needed to avoid unexpected changes. - Run load tests if possible: simulate concurrent users or multiple sessions to see if any race conditions or performance bottlenecks appear.

Infrastructure Choices: - LangGraph SaaS vs Self-Host: LangChain mentioned a LangGraph platform with 1-click deploy ¹¹⁶ ¹¹⁷. If available, that might handle scaling and state management for you.

Alternatively, you can deploy on your own environment. For self-host: - Use a web framework (FastAPI, Flask) to expose an API endpoint or WebSocket that clients can connect to for the chat. - Ensure the server process can handle multiple threads or use a job queue if some tasks are long. Since LangGraph can checkpoint, you could even design it so long tasks are done asynchronously and client polls or is notified when done. - If using persistent memory (Postgres, etc.), ensure that's deployed and backed-up as needed. - **Stateful vs Stateless APIs:** Because LangGraph can store state by thread_id, you can run the agent statelessly in the API – meaning each call provides the thread_id and you pull state from the store. This allows multiple instances of your service to share conversation state via the DB (so you can load balance without session affinity). - **Scaling LLM calls:** For heavy load, you might need to scale the LLM backend. If using OpenAI API, that's cloud-based so just watch rate limits. If using a local model (like running on GPUs), you might need to distribute requests across multiple GPU servers. Some orchestration or queue might be required if model inference is the bottleneck. - **Horizontal scaling of agent service:** The durable and persistent design of LangGraph means you can spin up multiple agent service instances. Each instance can handle requests, and as long as they point to the same persistence store, any can resume a conversation from checkpoint. This decouples memory from a single process's memory.

Reliability and Monitoring: - Use health checks for your agent service (maybe a simple endpoint that does a quick LLM call or memory check). - Monitor latency of responses. If it spikes, possibly an external API (like a tool) is slow – check logs. - Log important events: e.g., each time an agent uses a tool or ends a conversation, log it (to a file or observability system) so you can audit or analyze usage. - Use LangSmith's monitoring in production mode. Possibly integrate it to send you alerts if an agent's behavior deviates (they might have features like expected distributions of outputs). - Catch exceptions around agent.invoke and agent.stream. If an error occurs, you might attempt to resume from last checkpoint or at least fail gracefully by telling the user “Sorry, something went wrong” and logging the error for devs. - Use **fault tolerance:** If an LLM API fails (network issue), you can implement a retry mechanism (LangChain often has retry in its API wrapper). The durable execution means you can safely retry an LLM node without messing state if needed.

Cost management in production: Set up usage tracking per user. Perhaps limit free usage or implement billing if it's a paid service. Ensure your system doesn't accidentally run away (like a malicious user making the agent loop infinitely). The checks in module 9 help prevent that.

Capstone Use Case 1: Conversational Customer Support Agent

Imagine deploying an AI agent for an e-commerce site's support. It needs to: - Answer FAQs (using knowledge base documents). - Handle order tracking (maybe by calling a tracking API). - Handle returns or refunds (maybe requiring human approval for expensive items). - Chat naturally with users.

Using LangGraph: - **Tools & Retrieval:** Integrate a retrieval tool (vector search over FAQ docs) for general questions. Tools to query order database or trigger refund in system. - **Memory:** short-term to hold the conversation, long-term maybe to recall user's past issues if any (so it can say “Welcome back, I see you contacted us about a return last month” if appropriate). - **HITL:** For a refund above \$100, interrupt and ask human manager to approve (pattern from Module 6). - **Streaming:** As it answers or waits on API, stream typing indicator or partial answer for user experience. - **Multi-agent (optional):** Possibly have a separate agent persona for technical issues vs policy issues, and a supervisor routing to them. But a simpler approach is one agent with tools.

All modules come into play: - Foundations: Represent the workflow: maybe start with user query -> decide if needs knowledge base or action -> do it -> formulate answer. - State & Control: Graph with branches (if question is about order status -> call order API, if general -> retrieve FAQ). - Tools:

order_api_tool, faq_search_tool, refund_tool. - Persistence: use checkpoint to maintain conversation through multiple turns (very important in chat). - Memory: maybe store if user is VIP or past interaction summary to personalize responses. - HITL: certain flows escalate to human (guardrail if user is very unhappy or asks for manager). - Observability: trace how it's performing, ensure it's following refund policies etc. - Cost control: likely uses moderate-sized models (maybe GPT-3.5) for cost, and limits length of each response with summarization of context if needed.

This agent, once built and tested, could handle a large volume of support queries 24/7, with humans stepping in only when necessary – illustrating the power of combining automation with oversight.

Capstone Use Case 2: AI Research Assistant (Multi-Agent)

Consider an agent that helps a user write a research report: - It can search the web and gather information. - Summarize findings. - Draft sections of the report. - Critique and revise its drafts.

This can be naturally modeled with multiple agents or phases: 1. A Search Agent (uses tool to search internet). 2. A Summarizer Agent (reads retrieved docs, summarizing key points). 3. A Writer Agent (writes a draft based on outline and summaries). 4. A Reviewer Agent (analyzes draft and suggests improvements). 5. Loop between writer and reviewer for a few iterations.

Using LangGraph: - You could set up each as a subgraph or tool. Perhaps a main Orchestrator agent coordinates: first do search -> then summarization -> then plan outline -> then draft -> then critique -> if critique suggests more research, loop back to search, etc. This is complex but manageable by a graph with conditional edges (critique outcome decides next step). - Memory: store intermediate outputs (like content of found articles in long-term store so it doesn't lose them). - Multi-agent: The Reviewer could be a distinct prompt (more critical tone), the Writer a different prompt (creative tone). We can either combine them in one LLM with role instructions or use two LLM calls. Possibly hierarchical: top-level ensures the cycle doesn't repeat too many times.

This encompasses: - Advanced control flow (Module 2) with loops and conditions. - Tools for search (Module 3). - Persistence of state across long process (Module 4). - Long-term memory if user runs this agent in multiple sessions (store what it already researched yesterday). - Possibly HITL if the agent should ask the user for clarification or feedback at times ("Is this draft okay? Shall I add more on topic X?"). - Observability to debug any stuck loops or poor references. - Safety: If researching open web, guard against dangerous content incorporation, perhaps filter sources. - Cost control: Web search might return a lot of text – limit how much to feed into LLM at once (summarize sources one by one rather than giant context).

The output is a well-researched essay. This capstone shows LangGraph orchestrating many pieces: the reasoner, the tool user, the writer, the critic – essentially an entire team simulated.

Deployment Considerations for Capstones

For something like the above research assistant, running on server with internet access is key (for search). May also want GPU for summarizing lots of text (maybe use OpenAI models via API to offload heavy NLP tasks).

One might deploy it as: - A web app where user enters topic, then they can watch as the agent prints progress (like "Found 5 relevant articles. Summarizing... Drafting... Done."). - This could be packaged in a container, using LangChain's capabilities along with LangGraph's orchestration.

Wrap-up: Deploying an LangGraph agent involves not just the code but the ecosystem: keys, scaling, monitoring, and user interface. Each use case will have its nuances, but if you've built your agent following the modules, you have separation of concerns (tools, memory, control logic), making it easier to adjust and fix issues as they arise in production.

With careful design and testing, LangGraph agents can be **production-ready**, bringing the power of LLMs to real applications reliably ¹⁴⁰ ¹⁴¹. And because LangGraph is part of the LangChain suite, you can continue to leverage improvements in the ecosystem (like newer models, better memory stores, evaluation techniques) by updating components without rewriting your overall logic.

Suggested Exercise: Consider one of the capstone scenarios (customer support or research assistant) and draw a diagram of its LangGraph workflow (nodes and edges). Identify at least 3 points in the flow where you would add monitoring or guardrails in a production setting (for example: after knowledge retrieval, verify at least one source was found; or before executing an action, confirm it's within allowed policy). Discuss how you would deploy it (what API or interface, how to scale, etc.). This thought exercise solidifies your understanding of applying all the modules in a cohesive, real-world solution.

Sources: This handbook consolidated concepts from the LangGraph documentation and external resources to provide a thorough guide. Key references include the LangGraph docs on memory, tools, and multi-agent patterns ⁵ ¹² ¹²⁵, insights from Medium articles on LangGraph's capabilities ¹⁵ ²², and LangChain's official forums and blogs discussing best practices ¹³⁴ ¹⁴². By following the patterns and practices outlined, developers can confidently build and deploy sophisticated AI agents with LangGraph.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ³⁷ ³⁸ ³⁹ ⁴⁰ ¹⁰⁹ ¹¹⁰ ¹¹⁵

Building AI agent systems with LangGraph | by Vishnu Sivan | The Pythoneers | Medium

<https://medium.com/pythoneers/building-ai-agent-systems-with-langgraph-9d85537a6326>

¹⁷ ¹⁸ ⁴³ ⁶⁵ ¹³⁵ ¹³⁶ ¹⁴² LangGraph

<https://langchain-ai.github.io/langgraph/>

²⁵ ²⁶ ¹¹⁸ ¹¹⁹ ¹²⁰ ¹²¹ ¹²² ¹²³ ¹²⁴ ¹²⁵ ¹²⁶ ¹²⁷ ¹²⁸ ¹²⁹ ¹³⁰ ¹³¹ ¹³² ¹³³ Overview

https://langchain-ai.github.io/langgraph/concepts/multi_agent/

²⁷ ²⁸ ²⁹ ³⁰ ³¹ ³² ³³ ³⁴ ³⁵ ³⁶ Overview

<https://langchain-ai.github.io/langgraph/concepts/tools/>

⁴¹ ⁴² Overview

<https://langchain-ai.github.io/langgraph/concepts/time-travel/>

⁴⁴ ⁴⁵ ⁹⁵ ⁹⁶ ⁹⁷ ⁹⁸ ⁹⁹ ¹⁰⁰ ¹⁰¹ ¹⁰² Overview

https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/

⁴⁶ ⁴⁷ ⁴⁸ ⁴⁹ ⁵⁰ ⁵¹ ⁵² ⁶³ ⁶⁴ ⁶⁶ ⁶⁸ ⁶⁹ ⁷⁰ ⁷¹ ⁷² ⁷³ ⁷⁴ ⁷⁵ ⁷⁶ ⁷⁷ ⁷⁸ ⁷⁹ ⁸⁰ ⁸¹ ⁸² ⁸³ ⁸⁴ ⁸⁵ ⁸⁶
⁸⁷ ⁸⁸ ⁸⁹ ⁹⁰ ⁹¹ ⁹² ⁹³ ⁹⁴ Add memory

<https://langchain-ai.github.io/langgraph/how-tos/memory/add-memory/>

⁵³ ⁵⁴ ⁵⁵ ⁵⁶ ⁵⁷ ⁵⁸ ⁵⁹ Use time travel

https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/time-travel/

⁶⁰ ⁶¹ ⁶² ⁶⁷ LangGraph & Redis: Build smarter AI agents with memory & persistence | Redis

<https://redis.io/blog/langgraph-redis-build-smarter-ai-agents-with-memory-persistence/>

103 104 137 How to get token cost from a langGraph based implemented Openai Model · langchain-ai langchain · Discussion #24683 · GitHub

<https://github.com/langchain-ai/langchain/discussions/24683>

105 106 108 116 117 134 138 139 140 141 LangGraph

<https://www.langchain.com/langgraph>

107 111 113 114 Overview

<https://langchain-ai.github.io/langgraph/concepts/tracing/>

112 Calculate token-based costs for traces | 🦄 LangSmith

https://docs.smith.langchain.com/observability/how_to_guides/calculate_token_based_costs